

Dot Product Microbenchmarks: C, MKL, and Python Performance on the Roofline

Nrushad Joshi
Columbia University
naj2139@columbia.edu

Abstract—We benchmark five implementations of a single-precision dot product, ranging from a simple C loop to Intel MKL and a pure Python loop, and place every result on a roofline model. Each implementation is measured at $N = 10^6$, where the working set largely fits in cache, and at $N = 3 \times 10^8$, where the data streams from main memory. With an arithmetic intensity of 0.25 FLOP/byte, every implementation is memory bound, so they differ only in how close they come to the bandwidth roof. We find that optimized libraries come closest to the attainable performance and produce accurate results, while dependent additions and interpreter overhead hold back the simpler loops.

I. INTRODUCTION

The dot product is one of the simplest numerical kernels. Because it performs minimal arithmetic per byte of data, it is a standard example of a memory-bound computation. It is also a core building block of many larger workloads, such as matrix multiplication and neural network layers, so how fast it runs depends less on the processor’s compute power and more on how quickly data can be moved from memory. In this report, we compare five implementations of the dot product in C and Python, measure how close each one comes to the performance limit predicted by the roofline model, and check how accurate each result is.

II. BACKGROUND

A. The Roofline Model

The roofline model bounds the attainable performance P of a kernel by

$$P \leq \min(P_{\text{peak}}, I \times BW), \quad (1)$$

where P_{peak} is the peak compute performance, BW is the peak memory bandwidth, and I is the arithmetic intensity in FLOPs per byte moved. Kernels with I below the ridge point P_{peak}/BW are memory bound, and kernels above it are compute bound. We use a reference machine with $P_{\text{peak}} = 200$ GFLOP/s and $BW = 30$ GB/s, which places the ridge point at $200/30 \approx 6.67$ FLOP/byte.

B. Floating-Point Precision

A float32 value has 32 bits: 1 sign bit, 8 exponent bits, and 23 mantissa bits, giving 24 bits of precision. A number is stored as $1.m \times 2^e$, so the gap between neighboring representable numbers is 2^{e-23} , which doubles at every power of two. As a result, float32 can represent every integer exactly only up to $2^{24} = 16,777,216$.

III. IMPLEMENTATIONS

All five implementations compute $R = \sum_{i=0}^{N-1} A_i B_i$ on float32 inputs:

- **C1 (C loop):** a single loop with one float32 accumulator, compiled with `gcc -O3`.
- **C2 (C unrolled $\times 4$):** the same loop unrolled by four, summing four products before each update of the accumulator.
- **C3 (C + Intel MKL):** a call to `cblas_sdot` from Intel MKL.
- **C4 (Python loop):** an element-by-element Python loop over NumPy arrays, casting each element to a Python float.
- **C5 (Python + NumPy):** a call to `numpy.dot`.

IV. EXPERIMENTAL SETUP

All benchmarks were run on a Google Cloud e2-standard-8 VM (8 vCPUs, 32 GB RAM, x86-64 E2 platform) running Debian 12 in us-central1-a. The C programs were compiled with GCC 12 using `-O3 -Wall`, and C3 was linked against Intel oneAPI MKL. The Python programs used Python 3 with NumPy.

Each program takes the vector length N and a repetition count as arguments. We use $N = 10^6$ with 1000 repetitions and $N = 3 \times 10^8$ with 20 repetitions, with every element initialized to 1.0. Only the dot-product call is timed, using `clock_gettime(CLOCK_MONOTONIC)` in C and `time.perf_counter()` in Python. To prevent dead-code elimination, the result is stored in a volatile variable and printed.

We discard the first half of the repetitions, because the early ones include warm-up costs such as library initialization, page faults, and cold caches. The reported time $\langle T \rangle$ is the mean of the remaining repetitions, from which we compute bandwidth $B = 8N/\langle T \rangle$ and throughput $F = 2N/\langle T \rangle$. Each configuration was run three times, and all reported values are averages of these runs.

V. RESULTS

A. Roofline Analysis

Every implementation performs 2 FLOPs (one multiply and one add) for every 8 bytes read (two float32 values), giving an arithmetic intensity of 0.25 FLOP/byte. The ridge point of the reference machine is $200/30 \approx 6.67$ FLOP/byte, which

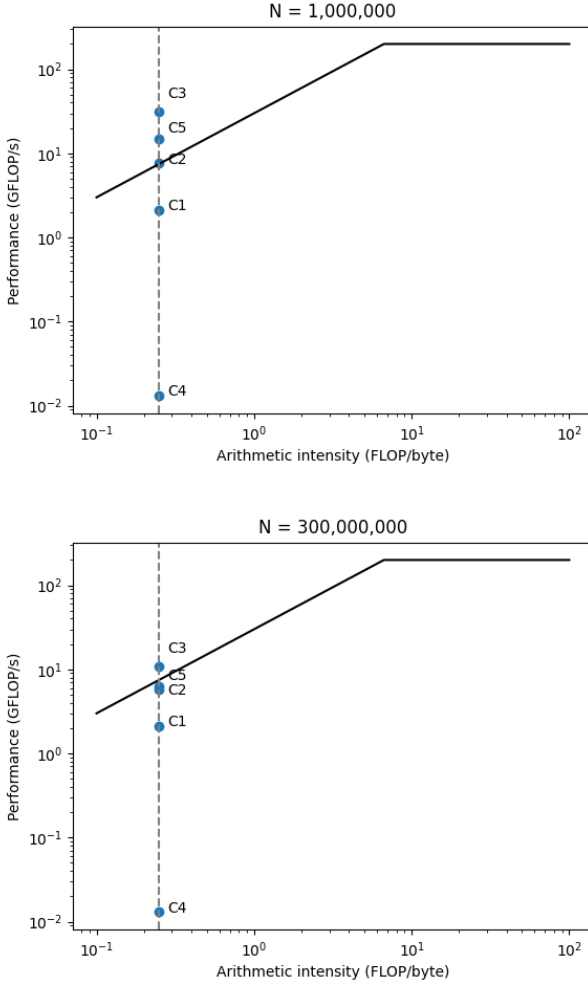


Fig. 1: Roofline model (peak 200 GFLOP/s, 30 GB/s) with all five implementations at arithmetic intensity 0.25 FLOP/byte, for $N = 10^6$ (top) and $N = 3 \times 10^8$ (bottom).

separates memory-bound kernels (to its left) from compute-bound kernels (to its right). Since 0.25 is far to the left of the ridge point, the processor finishes its arithmetic long before the data arrives from memory, so all five implementations are memory-bandwidth limited, and the roofline caps their performance at $30 \times 0.25 = 7.5$ GFLOP/s. The measurements in Fig. 1 then show how close each implementation comes to this limit.

C1 underperforms the roof because each addition must wait for the previous one to finish, so the loop is limited by this dependency rather than by memory. C4 underperforms the most, because every element goes through the Python interpreter, which adds a large overhead per iteration.

C3 at both sizes and C5 at $N = 10^6$ lie above the roof. This does not contradict the model: the 30 GB/s reference bandwidth is lower than what the VM actually delivers. At $N = 10^6$, the two arrays (8 MB) largely fit in cache, which is much faster than main memory. At $N = 3 \times 10^8$, MKL uses

multiple cores to read from memory at about 44 GB/s, above the reference peak.

B. Performance Comparison

TABLE I: Performance at $N = 3 \times 10^8$ (average of three runs).

	Implementation	$\langle T \rangle$ (s)	GB/s	GFLOP/s	Speedup
C1	C loop	0.2823	8.500	2.125	1.00
C2	C unrolled $\times 4$	0.1034	23.202	5.801	2.73
C3	C + Intel MKL	0.0549	43.697	10.924	5.14
C4	Python loop	47.9825	0.050	0.013	0.006
C5	Python + NumPy	0.0945	25.384	6.346	2.99

Table I compares the implementations at $N = 3 \times 10^8$, using the C loop as the baseline. C1 and C4 implement the exact same sequential algorithm. However, Python is an interpreted language and does not compile to machine code, so C4 is about $170\times$ slower than C1, making it the slowest of the five. C1 is the second slowest.

C2 is still sequential, but it performs four independent multiplications and adds their results in each iteration. This gives fewer iterations and fewer updates to the accumulator, so the processor can overlap more work, resulting in a $2.7\times$ speedup over C1 with no change in memory traffic.

C5 is faster than both sequential C programs, at $3.0\times$ over C1, because `numpy.dot` hands the whole array to an optimized BLAS library in a single call, paying Python’s overhead once rather than once per element. This makes it about $500\times$ faster than C4. C3 is the fastest at $5.1\times$ over C1, because MKL’s `cblas_sdot` is tuned for the processor and combines SIMD instructions, multiple accumulators, and multithreading.

C. Numerical Accuracy

TABLE II: Computed dot product; the analytical result is $R = N$.

	Implementation	$N = 10^6$	$N = 3 \times 10^8$
C1	C loop	1,000,000	16,777,216
C2	C unrolled $\times 4$	1,000,000	67,108,864
C3	C + Intel MKL	1,000,000	300,000,000
C4	Python loop	1,000,000	300,000,000
C5	Python + NumPy	1,000,000	300,000,000

Since every element is 1.0, the exact result is $R = N$. At $N = 10^6$, all five implementations are exact, because 10^6 is below 2^{24} . At $N = 3 \times 10^8$, only three are (Table II).

C1 adds 1 to a single float32 accumulator. Once the sum reaches $2^{24} = 16,777,216$, the gap between representable numbers grows to 2, so adding 1 is rounded away and the sum stops increasing. C2 adds 4 per iteration, so it keeps growing past 2^{24} until it reaches $2^{26} = 67,108,864$, where the gap becomes 8 and adding 4 is also rounded away.

C4 is exact because it accumulates in 64-bit Python floats, which represent every integer exactly up to 2^{53} . C3 and C5 are exact because BLAS splits the sum into many partial sums across SIMD lanes and threads; each partial sum stays below 2^{24} , and combining them at the end loses nothing.

VI. CONCLUSION

All five dot-product implementations are memory bound, so the differences between them come from how efficiently each one uses the available bandwidth. The naive C loop is held back by its chain of dependent additions, which unrolling partly removes. Optimized BLAS libraries go further by combining SIMD, multiple accumulators, and multithreading, while a pure Python loop is dominated by interpreter overhead. The same techniques that make the library kernels fast, splitting one long sum into many independent partial sums, also make them accurate, avoiding the precision loss of a single float32 accumulator.

ACKNOWLEDGMENT

This work was completed as part of COMS 6998: High Performance Machine Learning at Columbia University, taught by Dr. Kaoutar El Maghraoui. The assignment provided the dot-product function bodies (`dp` and `dpunroll` in C, `bdp` using MKL's `cblas_sdot`, and `dp` in Python), the NumPy array initialization, and the required output format. The benchmark harnesses, timing code, build setup, and plotting code were developed by me with the assistance of ChatGPT and Claude. I also used these tools to review my code, understand concepts such as floating-point precision and the roofline model, generate this report, and edit the writing. All benchmarks were run by me, and all results are from my own measurements.